

CI/CD + Infrastructure Pipeline Using Jenkins Clean, Simple & Real-World Breakdown

Modern DevOps pipelines don't just build code—they also manage infrastructure.

Here is a **clear, production-grade explanation** of how CI, CD, and Infra pipelines work together using Jenkins, along with environment promotion (Dev → QA → Prod), IaC, and a clean example Jenkinsfile.

1. High-Level Pipeline Flow

Developer pushes code →

Jenkins CI builds & tests →

Artifact/image stored in registry →

CD pipeline deploys to **Dev**, then **QA**, then **Prod** after approvals →

Infra changes run through a **Terraform pipeline**.

2. Git Branching & Promotion Model

Common branch structure:

- **feature/** → developer work
- **develop** → auto deploy to **Dev**
- **release/qa** → for QA testing
- **main/master** → protected, for Production only

Two real approaches:

- **Trunk-Based Development** — fast, frequent merges
 - **GitFlow** — structured for larger teams
-

3. Key Components Used

Jenkins (Multibranch / Pipeline-as-Code)

Docker + Registry (ECR, GCR, Artifactory, Nexus)

Terraform / Ansible

SonarQube (code quality)
Smoke, Integration, E2E tests
Secrets Manager / Vault
Monitoring + Rollback (Prometheus, Grafana)

4. CI Pipeline (Every Push / Every PR)

CI ensures quick feedback.

Typical stages:

1. **Checkout code**
2. **Install dependencies**
3. **Build / Package**
4. **Unit tests**
5. **Static analysis + Security scans**
6. **Build Docker image & push to registry**
7. **Publish reports (JUnit, coverage)**

Runs on **every PR + develop branch**.

5. CD Pipeline (Dev → QA → Prod)

Dev Deployment (fully automated)

1. Pull latest artifact/image
2. Deploy to Dev environment
3. Smoke tests / quick integration tests
4. Auto rollback on failure
5. Notify team

QA Deployment (semi-automated)

1. Manual or automated promotion
2. Deploy to QA
3. Integration + E2E tests
4. Security scans / load tests
5. QA sign-off

Prod Deployment (strict & protected)

1. Mandatory manual approval
 2. Canary / Blue-Green deployment
 3. Post-deploy health checks
 4. Rollback if failure threshold exceeded
 5. Audit + notifications
-

6. Infrastructure Pipeline (Terraform/Ansible)

A separate pipeline (recommended):

1. `terraform init`
2. `terraform plan` (shared in PR)
3. Manual approval
4. `terraform apply`
5. Smoke test infra
6. Remote state + locking for safety

Infra changes move first → app deploys follow.

7. Clean & Simple Jenkinsfile Example

```
pipeline {
  agent any

  environment {
    REGISTRY = "myregistry/myapp"
    IMAGE_TAG = "${env.BRANCH_NAME}-${env.BUILD_NUMBER}"
  }

  stages {
    stage('Checkout') { steps { checkout scm } }

    stage('Build') {
      steps {
        sh './mvnw -DskipTests package'
        archiveArtifacts 'target/*.jar'
      }
    }

    stage('Unit Tests') {
      steps {
        sh './mvnw test'
        junit 'target/surefire-reports/*.xml'
      }
    }
  }
}
```

```

}

stage('Static Analysis') {
  steps {
    withCredentials([string(credentialsId: 'sonar-token', variable:
'TOKEN')]) {
      sh "./mvnw sonar:sonar -Dsonar.login=$TOKEN"
    }
  }
}

stage('Build & Push Image') {
  steps {
    sh "docker build -t ${REGISTRY}:${IMAGE_TAG} ."
    withCredentials([usernamePassword(credentialsId: 'docker-creds',
usernameVariable: 'U', passwordVariable: 'P')]) {
      sh "echo $P | docker login -u $U --password-stdin myregistry"
      sh "docker push ${REGISTRY}:${IMAGE_TAG}"
    }
  }
}

stage('Deploy to Dev') {
  steps {
    sh "kubectl set image deployment/myapp myapp=${REGISTRY}:${IMAGE_TAG} -n
dev"
    sh "./scripts/smoke-tests.sh dev"
  }
}

stage('Promote to QA') {
  steps { input message: "Promote to QA?" }
}

stage('Deploy to QA') {
  steps {
    sh "kubectl set image deployment/myapp myapp=${REGISTRY}:${IMAGE_TAG} -n
qa"
    sh "./scripts/e2e-tests.sh qa"
  }
}

stage('Approve Prod Deployment') {
  steps { input message: "Deploy to Production?" }
}

stage('Deploy to Prod') {
  steps {
    sh "kubectl apply -f k8s/canary.yaml -n prod"
    sh "./scripts/post-deploy-checks.sh prod"
  }
}

post {
  success { echo 'Pipeline Succeeded!' }
  failure { echo 'Pipeline Failed!' }
}
}

```

8. Dev vs QA vs Prod — What's Different?

Environment	Purpose	Deployment	Tests
Dev	Fast validation	Auto	Smoke, quick integration
QA	Quality verification	Semi-auto	E2E, regression, security
Prod	Customer traffic	Manual approval	Health checks, monitoring

9. Testing Strategy Mapping

- **Unit tests** → CI
 - **Integration tests** → Dev
 - **E2E tests** → QA
 - **Load/Performance** → QA/Staging
 - **Security tests** → CI + QA
 - **Health checks** → Prod
-

10. Safe Deployment Strategies

Blue/Green

Canary

Rolling

Auto Rollback

Pick based on traffic and downtime tolerance.

11. Rollback + Monitoring Best Practices

- Always tag images with build number
 - Keep last 10 versions easily deployable
 - Monitor CPU, memory, errors, latency after deploy
 - Rollback automatically if metrics degrade
-

12. Best Practices Summary

Use multibranch pipelines

Keep CI fast (fail early)

- Keep CD slow & safe (approvals + gates)
- Move infra via a separate Terraform pipeline
- Store no secrets in code
- Use immutable artifacts
- Prefer canary for production
- Document everything